

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: Observation Task-02

Student Name: Ganji Venkata Vijay Anudeep

Roll No.: A24126552081 Date: 30-08-2026

1. TITLE

Develop a Dynamic Student Profile Generator Web Application Using HTML, CSS, and Object-Oriented JavaScript.

2. AIM

To develop a client-side web application using HTML, CSS, and JavaScript that accepts student information through an input form, validates the data, and dynamically generates and displays a student profile card using Object-Oriented Programming (OOP) concepts.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the structure of HTML forms and input elements.
- To style web elements and create responsive layouts using CSS.
- To implement Object-Oriented Programming (OOP) in JavaScript through classes and constructors.
- To perform DOM (Document Object Model) selection and manipulation.
- To implement event handling and user input validation using JavaScript.
- To dynamically create and render HTML elements based on user input.

4. THEORY

- HTML provides the basic structure of the web page, defining input fields (text, numbers) and containers for the dynamic content.

- CSS is used to style the application, utilizing features like linear gradients, flexbox for alignment, hover effects, and CSS animations (keyframes) for smooth rendering.
- JavaScript provides the functional logic for the application:
- Classes (class Student): Allow encapsulation of student properties (name, roll number, department, CGPA) into reusable object blueprints.
- DOM Selection: Methods like `document.getElementById()` are used to target specific HTML elements.
- Event Listeners: `addEventListener()` triggers the profile generation process when the user clicks the submission button.
- Validation: Ensures that input fields are not empty and numerical limits (like CGPA constraints) are enforced before proceeding.
- DOM Manipulation: Methods like `document.createElement()` and `appendChild()` allow the script to build and display new HTML elements dynamically on the webpage without reloading.

5. ALGORITHM / PROCEDURE

1. Create a project folder containing three files: `index.html`, `style.css`, and `script.js`.
2. Define the structural layout in the HTML file, including input fields for Name, Roll Number, Department, CGPA, and a Generate button.
3. Add a placeholder `div` element to hold the dynamically generated profile card.
4. Write CSS rules to style the form, inputs, buttons, error messages, and define an animation for the profile card rendering.
5. In the JavaScript file, define a `Student` class containing a constructor for the student properties.

6. Select the necessary DOM elements (inputs, button, error message, profile container) using their respective IDs.
7. Attach a click event listener to the Generate button.
8. Extract and trim the values from the input fields upon a button click.
9. Check if any fields are empty. If so, display an error message and halt execution.
10. Validate the CGPA to ensure it is a number between 0 and 10. Display an error if it fails validation.
11. Instantiate a new `Student` object passing the sanitized inputs.
12. Create a function `renderProfile(student)` to clear any existing profile and construct a new profile card using `document.createElement()`.
13. Append the newly created text nodes, labels, and badges to the profile card container, and append the card to the main webpage.
14. Perform test cases in the web browser to verify correct execution and validation.

6. PROGRAM

index.html

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Profile Generator</title>
  <!-- Main External Stylesheet -->
  <link rel="stylesheet" href="./style.css">
</head>
<body>

  <!-- Main Card Container -->
  <div class="container">
    <h1>Student Profile Generator</h1>
```

```

<!-- User Input Form Section -->
<label for="nameInput">Name</label>
<input type="text" id="nameInput" placeholder="e.g. Ravi Kumar">

<label for="rollInput">Roll Number</label>
<input type="text" id="rollInput" placeholder="e.g. A23126552001">

<label for="deptInput">Department</label>
<input type="text" id="deptInput" placeholder="e.g. CSE (AI & ML)">

<label for="cgpaInput">CGPA</label>
<input type="number" id="cgpaInput" step="0.01" min="0" max="10"
placeholder="e.g. 8.42">

<!-- Action Button -->
<button id="generateBtn">Generate Student Profile</button>

<!-- Error Message Display -->
<p class="error" id="errorMsg"></p>

<!-- Dynamic Profile Output Container -->
<div id="profileContainer"></div>
</div>

<!-- Main Application Script -->
<script src="./script.js"></script>
</body>
</html>

```

style.css

CSS

```

/* =====
Base Layout & Global Styles
=====
*/
body {
  font-family: 'Segoe UI', Tahoma, Verdana, sans-serif;
  background: linear-gradient(135deg, #1e293b, #0f172a);
  min-height: 100vh;    margin: 0;
  display: flex;
  align-items: center;
  justify-content: center;

```

```

padding: 30px 15px;
box-sizing: border-box;
}

/* =====
Form Container & Input Controls
===== */
.container {
background: #ffffff;
width: 100%;
max-width: 420px;
border-radius: 14px;
padding: 28px 26px;
box-shadow: 0 20px 45px rgba(0, 0, 0, 0.35);
}

h1 {
text-align: center;
font-size: 1.4rem;
color: #1e293b;
margin-bottom: 22px;
margin-top: 0;
}

label {
display: block;
font-size: 0.85rem;
font-weight: 600;
color: #334155;
margin-bottom: 4px;
margin-top: 14px;
}

input {
width: 100%;
padding: 9px 11px;
border: 1px solid #cbd5e1;
border-radius: 7px;
font-size: 0.95rem;
outline: none;
box-sizing: border-box;
transition: border-color 0.2s;
}

```

```
input:focus {
  border-color: #6366f1;
}
```

```
button {
  width: 100%;
  margin-top: 22px;
  padding: 11px;
  background: #6366f1;
  color: white;
  border: none;
  border-radius: 8px;
  font-size: 1rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.2s, transform 0.1s;
}
```

```
button:hover {
  background: #4f46e5;
}
```

```
button:active {
  transform: scale(0.98);
}
```

```
.error {
  color: #dc2626;
  font-size: 0.82rem;
  margin-top: 10px;
  text-align: center;
  min-height: 18px;
}
```

```
/* =====
Dynamically Generated Profile Card
=====
*/
```

```
.profile-card {
  margin-top: 24px;
  background: #f8fafc;
  border: 1px solid #e2e8f0;
  border-left: 5px solid #6366f1;
  border-radius: 10px;
  padding: 18px 20px;
```

```
    animation: fadeIn 0.35s ease;
}

/* Fade-in Animation for Profile Card */
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(-8px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.profile-card h2 {
  margin: 0 0 12px 0;
  font-size: 1.1rem;
  color: #1e293b;
  border-bottom: 2px solid #6366f1;
  padding-bottom: 8px;
}

.profile-row {
  display: flex;
  justify-content: space-between;
  padding: 6px 0;
  font-size: 0.92rem;
  border-bottom: 1px dashed #e2e8f0;
}

.profile-row:last-child {
  border-bottom: none;
}

.profile-row .field-label {
  font-weight: 600;
  color: #475569;
}

.profile-row .field-value {
  color: #0f172a;
}
```

```
.cgpa-badge {
  display: inline-block;
  background: #6366f1;
  color: white;
  padding: 2px 10px;
  border-radius: 999px;
  font-weight: 600;
  font-size: 0.85rem;
}
```

script.js

JavaScript

```
/**
 * Represents a Student entity.
 */
class Student {
  constructor(name, rollNumber, department, cgpa) {
    this.name = name;
    this.rollNumber = rollNumber;
    this.department = department;
    this.cgpa = cgpa;
  }
}

// =====
// DOM Elements Selection
// =====
const nameInput = document.getElementById('nameInput');
const rollInput = document.getElementById('rollInput');
const deptInput = document.getElementById('deptInput');
const cgpaInput = document.getElementById('cgpaInput');
const generateBtn = document.getElementById('generateBtn');
const errorMsg = document.getElementById('errorMsg');
const profileContainer = document.getElementById('profileContainer');

// =====
// Event Listeners & Input Processing
// =====
generateBtn.addEventListener('click', () => {
  // Sanitize user inputs
  const name = nameInput.value.trim();
  const roll = rollInput.value.trim();
  const dept = deptInput.value.trim();
  const cgpa = cgpaInput.value.trim();
```

```

    // Validate empty fields
    if (!name || !roll || !dept || !cgpa) {
        errorMsg.textContent = 'Please fill in all fields before generating
the profile.';
        return;
    }

    // Validate CGPA range and numerical format
    if (isNaN(cgpa) || cgpa < 0 || cgpa > 10) {
        errorMsg.textContent = 'CGPA must be a number between 0 and 10.';
        return;
    }

    // Clear any active error message on successful validation
    errorMsg.textContent = '';

    // Instantiate Student object with formatted CGPA value
    const student = new Student(name, roll, dept,
parseFloat(cgpa).toFixed(2));

    // Render the card to the DOM
    renderProfile(student);
});

// =====
// UI Rendering Functions
// =====
/**
 * Builds and renders the profile card dynamically inside the DOM.
 * @param {Student} student - The student object containing profile data.
 */
function renderProfile(student) {
    // Clear previous card output
    profileContainer.innerHTML = '';

    // Create main card element
    const card = document.createElement('div');
    card.className = 'profile-card';

    // Create card heading
    const title = document.createElement('h2');
    title.textContent = 'Student Profile';
    card.appendChild(title);

```

```

// Schema mapping for profile key-value details
const fields = [
  { label: 'Name', value: student.name },
  { label: 'Roll No', value: student.rollNumber },
  { label: 'Department', value: student.department },
  { label: 'CGPA', value: student.cgpa, isCgpa: true }
];

// Build each data row dynamically
fields.forEach(field => {
  const row = document.createElement('div');
  row.className = 'profile-row';

  const labelSpan = document.createElement('span');
  labelSpan.className = 'field-label';
  labelSpan.textContent = field.label;

  const valueSpan = document.createElement('span');
  valueSpan.className = 'field-value';
  // Custom formatting for CGPA badge element
  if (field.isCgpa) {
    const badge = document.createElement('span');
    badge.className = 'cgpa-badge';
    badge.textContent = field.value;
    valueSpan.appendChild(badge);
  } else {
    valueSpan.textContent = field.value;
  }

  row.appendChild(labelSpan);
  row.appendChild(valueSpan);
  card.appendChild(row);
});

// Mount generated card into the DOM wrapper
profileContainer.appendChild(card);
}

```

7.CODE EXPLANATION

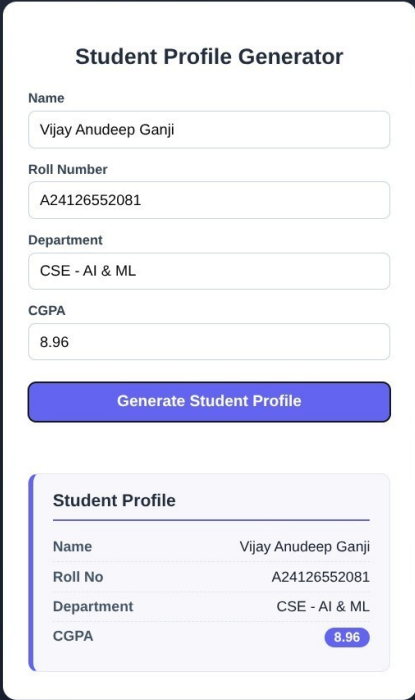
Code	Explanation
------	-------------

<code>class Student</code>	Defines the blueprint for creating a student object with properties.
<code>document.getElementById()</code>	Targets and selects HTML elements to allow manipulation in JavaScript.
<code>addEventListener('click', ...)</code>	Attaches a click event to the "Generate Student Profile" button to execute the code inside the callback.
<code>new Student(...)</code>	Creates a new instance of the <code>Student</code> object using the validated input values.
<code>document.createElement('div')</code>	Dynamically generates a new HTML <code>div</code> element via JavaScript.
<code>element.appendChild(...)</code>	Appends newly created elements (like the profile card or row) to their parent container.
<code>innerHTML = "</code>	Clears out any existing profile content before rendering a new one.

8.TEST CASES AND EXECUTION DATA

-- screenshots attched --

9.OUTPUT



The screenshot displays a web application titled "Student Profile Generator" on a dark blue background. The application consists of a form with four input fields: "Name" (containing "Vijay Anudeep Ganji"), "Roll Number" (containing "A24126552081"), "Department" (containing "CSE - AI & ML"), and "CGPA" (containing "8.96"). Below these fields is a blue button labeled "Generate Student Profile". Below the button is a "Student Profile" card. The card contains the following information:

Student Profile	
Name	Vijay Anudeep Ganji
Roll No	A24126552081
Department	CSE - AI & ML
CGPA	8.96

10.RESULT

Thus, a dynamic Student Profile Generator application was successfully developed using HTML, CSS, and Object-Oriented JavaScript. Client-side input validation was implemented, DOM selection and creation were executed successfully, and the logic was verified using multiple test cases.

